

Deep and Un-Deep Visual Inertial Odometry

RBE 549: P4

Pau Alcolea
Department of Robotics
Worcester Polytechnic Institute
palcolea@wpi.edu

Ben Orr
Department of Robotics
Worcester Polytechnic Institute
bcorr@wpi.edu

Abstract—In this paper we present a successful Python implementation of "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight" [1]. Using information-dense stereo vision, this method outperforms monocular approaches without compromising on computational efficiency.

I. PHASE 1: CLASSICAL VISUAL-INERTIAL ODOMETRY

A. Problem Statement

The goal of this project is to demonstrate a robust method for robot state estimation under various constraints. For small aerial robots, Visual Inertial Odometry (VIO) has the advantage of being lighter than lidar while maintaining performance in GPS-denied environments. One such significant work in VIO is "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight" by Sun et al. [1], which we present our implementation of here. This approach is a filter-based stereo VIO that uses the Multi-State Constraint Kalman Filter (MSCKF) [2], consisting of a combination of a camera and an Inertial Measurement Unit (IMU). It is referred to as the Stereo Multi-State Constraint Kalman Filter (S-MSCKF) [1].

To understand why the combination of these two sensors is imperative for an accurate and reliable reading, the fallbacks of the camera and IMU independently have to be considered. Stereo vision can be used for depth calculations through feature matching and a known baseline, but this method breaks down when the frames experience severe motion blur, as is the case in many robotics applications. In this case, we can compliment the camera readings with those of the IMU, as they are complimentary sensors. Where a camera fails: fast motion and motion blur, the IMU excels, and where the IMU falls short: long term stability and drifts, the camera can produce stable results.

B. Data

Following [1], our project was evaluated on a subset of EuRoC [3], a visual-inertial dataset collected on a micro aerial vehicle. Specifically, we used the "Machine Hall 01 easy" subset, which contains stereo images and synchronized IMU measurements alongside their ground-truth trajectories obtained from a Vicon motion capture system.

C. Software Overview

For this project, we implemented a subset of functions that form the core of the S-MSCKF. These include the following:

- `initialize_gravity_and_bias()`
- `batch_imu_processing()`
- `process_model()`
- `predict_new_state()`
- `state_augmentation()`
- `add_feature_observations()`
- `measurement_update()`

Each function will be described in greater detail below. A visual overview of the algorithmic flow can also be found in Figure 1.

D. Gravity and Bias Initialization

Even the highest quality IMUs will suffer from biases in measured quantities. To account for this in `initialize_gravity_and_bias()`, we began by calculating the bias for the gyroscope by averaging the first 200 data points taken when the robot was stationary. Likewise, we found the expected gravity vector from the averaged accelerometer measurements. This allowed us to orient the robot's coordinate system in accordance with what it was perceiving.

E. Batch IMU Processing

By nature, the camera gets frames at a much slower frequency than the IMU can read, which makes the combination of readings non-trivial. The way this is handled in `batch_imu_processing()` is to process all of the IMU readings in between camera-frames together. The timestamps for the readings are compared with the camera frame's, and each sensor reading is processed; updating the IMU state incrementally. This bridge between the higher-frequency IMU measurements and the lower-frequency camera frames makes their combination in section I-J

F. Process Model

The process model method implements the section III.A from Sun et al., [1], propagating the state estimate and its uncertainty forward. This process calculates the system matrices F and G from the corrected angular velocity and acceleration. The current bias estimates in the state are used for this correction to try to minimize the systematic error of IMUs, $\omega = \omega_m - \hat{b}_g$, $\hat{a} = a_m - \hat{b}_a$. $\mathbf{F}$, the continuous-time error-state matrix shows how the existing errors grow and change

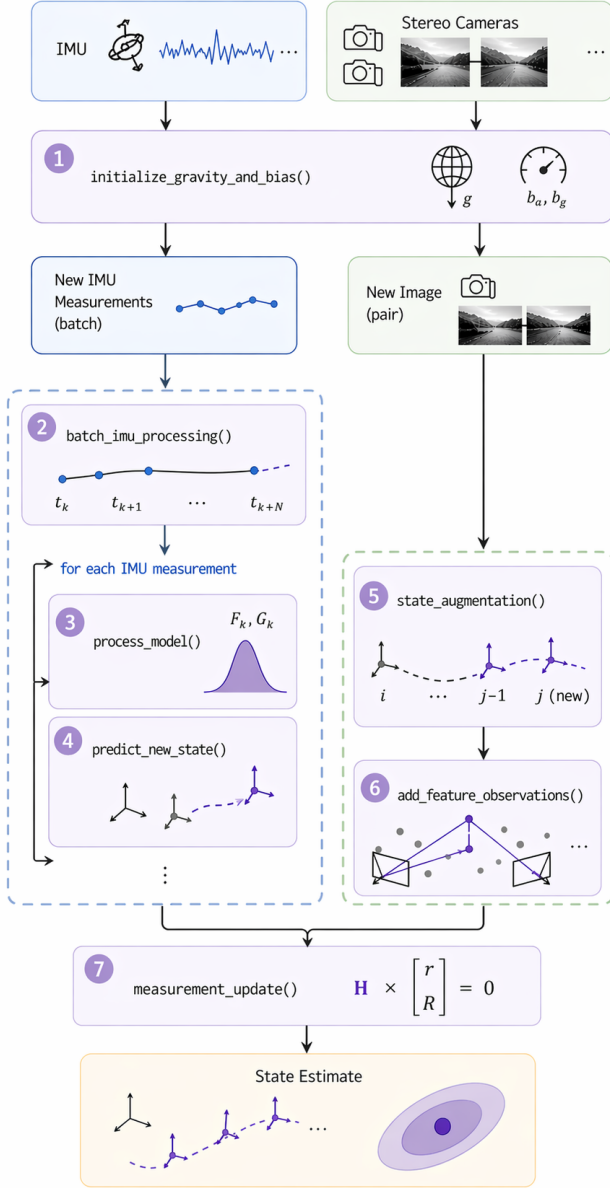


Fig. 1. High-level overview of the functions we implemented

over time, and how they are related between one another, i.e. $\mathbf{F}$ can show how a gyro bias error might cause an orientation to accumulate. Equation (1) shows the continues evolution of the state errors, showing how they propagate through the dynamics and how noise enters the system. In a similar way, $\mathbf{G}$, the noise input matrix, denotes how IMU noise is going to affect each part of the error state. The combination of these two matrices in eq. (1) is what can compute uncertainty properly and capture the IMU drift properly.

$$\dot{\mathbf{x}} = \mathbf{F} \mathbf{x} + \mathbf{G} \mathbf{n} \quad (1)$$

To use the calculated matrices, instead of performing matrix

exponential, which is extremely intensive to compute, we used a 3rd-order discrete-time approximation, eq. (2). This approximation is appropriate for the small changes in time dt , which is the case for our IMU, and is much more feasible computationally.

$$\Phi(t) \approx \mathbf{I} + \mathbf{F}\Delta t + \frac{(\mathbf{F}\Delta t)^2}{2!} + \frac{(\mathbf{F}\Delta t)^3}{3!} \quad (2)$$

The standard EFK linearizes the dynamics a opposed to updating the measurement, which causes the filter to gain incorrect information about unobservable directions: global yaw and global position. These are unobservable because they are properties that cannot be derived from the camera-IMU setup. This creates inconsistencies, that are fixed by enforcing an observability constraint, which modifies the state transition matrix so that unobserved properties remain null.

The IMU state covariance is calculated eq. (3), which accounts for noise that enters throughout the given interval, and propagated eq. (4) with the filter prediction. The propagation updates the noise through the new dynamics from the measurements.

$$Q_k = \int_{t_k}^{t_{k+1}} \Phi(t_{k+1}, \tau) G Q G^T \Phi(t_{k+1}, \tau)^T d\tau \quad (3)$$

$$P_{II}^{k+1|k} = \Phi_k P_{II}^{k|k} \Phi_k^T + Q_k \quad (4)$$

Finally, the camera covariance gets propagated with the obtained information, as it is still susceptible to uncertainty. The states for the camera are fixed poses from frames, so they don't accumulate process noise, that being said, the IMU uncertainty still affects those poses because of the augmentation step, section I-H.

G. New State Prediction

The goal of the `predict_new_state()` function is to update the IMU state (orientation $\mathbf{q}$, velocity $\mathbf{v}$, and position $\mathbf{p}$) using measurements from the gyroscope and accelerometer. We achieved this by implementing the following mathematical formulation:

Consider the IMU state dynamics to be a continuous non-linear system (eq. (5)). With the assumption that ω is constant over dt , we can integrate $\mathbf{q}$ with the closed-form solution in eq. (6), where the matrix Ω (eq. (7)) maps angular velocity ω to quaternion time derivatives.

$$\begin{aligned} \dot{\mathbf{q}} &= \frac{1}{2} \Omega(\omega) \mathbf{q}, \\ \dot{\mathbf{v}} &= R(\mathbf{q})^T \mathbf{a} + \mathbf{g}, \\ \dot{\mathbf{p}} &= \mathbf{v}, \end{aligned} \quad (5)$$

$$\mathbf{q}(t + \Delta t) = \left(\cos\left(\frac{\|\omega\|\Delta t}{2}\right) \mathbf{I} + \frac{\sin\left(\frac{\|\omega\|\Delta t}{2}\right)}{\|\omega\|} \Omega(\omega) \right) \mathbf{q}(t). \quad (6)$$

$$\Omega(\omega) = \begin{bmatrix} -[\omega]_{\times} & \omega \\ \omega^T & 0 \end{bmatrix}. \quad (7)$$

To approximate the system dynamics, we use the 4th order Runge-Kutta method. Applying eq. (8), derivative estimates are initialized with eq. (9), refined with eqs. (10) and (11), evaluated with eq. (12) and accumulated with eq. (13), which represents the updated velocity and position after one additional IMU measurement.

$$\mathbf{q}\left(t + \frac{\Delta t}{2}\right) = \exp\left(\frac{\Delta t}{2}\Omega(\omega)\right)\mathbf{q}(t), \quad (8)$$

$$\mathbf{a}_w(\mathbf{q}) = R(\mathbf{q})^T \mathbf{a} + \mathbf{g}$$

$$\begin{aligned} \mathbf{k}_1^p &= \mathbf{v}, \\ \mathbf{k}_1^v &= \mathbf{a}_w(\mathbf{q}) \end{aligned} \quad (9)$$

$$\begin{aligned} \mathbf{k}_2^p &= \mathbf{v} + \frac{\Delta t}{2}\mathbf{k}_1^v, \\ \mathbf{k}_2^v &= \mathbf{a}_w\left(\mathbf{q}_{\frac{1}{2}}\right) \end{aligned} \quad (10)$$

$$\begin{aligned} \mathbf{k}_3^p &= \mathbf{v} + \frac{\Delta t}{2}\mathbf{k}_2^v, \\ \mathbf{k}_3^v &= \mathbf{a}_w\left(\mathbf{q}_{\frac{1}{2}}\right) \end{aligned} \quad (11)$$

$$\begin{aligned} \mathbf{k}_4^p &= \mathbf{v} + \Delta t \mathbf{k}_3^v, \\ \mathbf{k}_4^v &= \mathbf{a}_w(\mathbf{q}_{\Delta t}) \end{aligned} \quad (12)$$

$$\begin{aligned} \mathbf{v}_{k+1} &= \mathbf{v}_k + \frac{\Delta t}{6}(\mathbf{k}_1^v + 2\mathbf{k}_2^v + 2\mathbf{k}_3^v + \mathbf{k}_4^v), \\ \mathbf{p}_{k+1} &= \mathbf{p}_k + \frac{\Delta t}{6}(\mathbf{k}_1^p + 2\mathbf{k}_2^p + 2\mathbf{k}_3^p + \mathbf{k}_4^p). \end{aligned} \quad (13)$$

H. State Augmentation

`state_augmentation()` is called when a new image is received. It records the pose of the camera and updates the state covariance matrix $\mathbf{P}$.

Camera pose is constructed by applying the known extrinsic transformation between the IMU and the camera, as in eqs. (14) to (16):

$$\mathbf{R}_W^C = \mathbf{R}_I^C \mathbf{R}_W^I \quad (14)$$

$$\mathbf{p}_C = \mathbf{p}_I + (\mathbf{R}_W^I)^T \mathbf{t}_C^I \quad (15)$$

$$\mathbf{x}_C = \begin{bmatrix} \mathbf{q}_C \\ \mathbf{p}_C \end{bmatrix} \quad (16)$$

where $\mathbf{R}_I^C$ is the orientation of the camera in the IMU frame, $\mathbf{t}_C^I$ is the translation from the IMU frame to the camera frame, $\mathbf{q}_C$ is the quaternion representation of $\mathbf{R}_W^C$, and $\mathbf{x}_C$ is the state of the camera.

With the newly-obtained camera pose information, we can augment state covariance matrix $\mathbf{P}$ to get $\mathbf{P}^+$ as follows:

$$\mathbf{P}^+ = \begin{bmatrix} \mathbf{P} & \mathbf{P}\mathbf{J}^T \\ \mathbf{J}\mathbf{P} & \mathbf{J}\mathbf{P}\mathbf{J}^T \end{bmatrix} \quad (17)$$

In eq. (17), Jacobian matrix $\mathbf{J} \in \mathbb{R}^{6 \times 21}$ is the mapping from the 21-dimensional IMU error-state to the new 6-dimensional camera error state. It is defined below (see also Appendix B of [1]):

$$\begin{aligned} \mathbf{J} &= [\mathbf{J}_I \quad 0_{6 \times 6N}] \\ \mathbf{J}_I &= \begin{bmatrix} \mathbf{R}_C^I & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 & \mathbf{0} \\ (\mathbf{R}_W^I)^T [\mathbf{t}_C^I]_{\times} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \end{aligned} \quad (18)$$

Because the new camera pose is calculated using the IMU pose, and the IMU is not error-free, the use of $\mathbf{J}$ is vital in obtaining $\mathbf{P}^+$ if it is to accurately track the highly-correlated uncertainties in the system.

I. Adding Feature Observations

After the state and covariance are updated with the new image's camera information, features in the image are recorded by `add_feature_observations()`.

While feature extraction happens elsewhere in the system, it is this function that handles the features for use in the S-MSCKF VIO algorithm. As seen in Algorithm 1, we iterate through all features and record observations of both new and known ones, enabling us to perform tracking across frames.

Algorithm 1 Add Feature Observations

Require: Feature messages $\mathcal{F}$, state server $\mathcal{S}$, map server $\mathcal{M}$
Ensure: Updated map server $\mathcal{M}$ and tracking rate r

```

1:  $N \leftarrow |\mathcal{M}|$ 
2:  $N_{\text{tracked}} \leftarrow 0$ 
3: for each feature  $f \in \mathcal{F}$  do
4:   if  $f.\text{id} \notin \mathcal{M}$  then
5:     Initialize new feature in map server  $\mathcal{M}$ :

```

$$\mathcal{M}[f.\text{id}] \leftarrow \text{Feature}(f.\text{id})$$

```

6:   else
7:      $N_{\text{tracked}} \leftarrow N_{\text{tracked}} + 1$ 
8:   end if
9:   Store observation  $\mathbf{z}$  in map server  $\mathcal{M}$ :

```

$$\mathcal{M}[f.\text{id}].\text{observations}[\mathcal{S}.\text{imu_state}.\text{id}] \leftarrow \mathbf{z}$$

```

10: end for
11: if  $N = 0$  then
12:    $r \leftarrow 1.0$ 
13: else
14:    $r \leftarrow \frac{N_{\text{tracked}}}{N}$ 
15: end if

```

J. Measurement Update

After measurement residuals have been assembled, `measurement_update()` applies the EKF correction step.

Given measurement residuals $\mathbf{r}$, and Jacobian matrix $\mathbf{H}$ obtained from tracked features, we first project the system (eq. (19)) into a reduced space via QR decomposition (see eqs. (20) and (21)) to obtain $\mathbf{r}'$ and $\mathbf{H}'$.

$$\mathbf{r} \approx \mathbf{H}\delta\mathbf{x} + \mathbf{n} \quad (19)$$

$$\begin{aligned} \mathbf{H} &= \mathbf{Q}\mathbf{R}, \\ \mathbf{Q}^T \mathbf{r} &\approx \mathbf{R}\delta\mathbf{x} + \mathbf{Q}^T \mathbf{n} \end{aligned} \quad (20)$$

$$\begin{aligned} \mathbf{r}' &= \mathbf{Q}^T \mathbf{r}, \\ \mathbf{H}' &= \mathbf{R} \end{aligned} \quad (21)$$

Next we compute the Kalman gain $\mathbf{K}$, which by mapping measurement errors in pixel space to errors in the state space, determines how much we should trust the measurement as opposed to the current state estimate. Consider eq. (22) where $\mathbf{P}$ is the state covariance matrix and $\mathbf{S}$ is the innovation covariance.

$$\mathbf{K} = \mathbf{P}\mathbf{H}'^T \mathbf{S}^{-1} \quad (22)$$

Defined in eq. (23), the innovation covariance matrix accounts for the uncertainty in the difference between predictions and actual measurements. Here, $\mathbf{R}'$ is a diagonal matrix known as the measurement noise covariance matrix (see eq. (24)). In our implementation, $\sigma = 0.035$.

$$\mathbf{S} = \mathbf{H}'\mathbf{P}\mathbf{H}'^T + \mathbf{R}' \quad (23)$$

$$\mathbf{R}' = \sigma^2 \mathbf{I} \quad (24)$$

The error in the state $\delta\mathbf{x}$ is the product of Kalman gain $\mathbf{K}$ and reduced measurement residuals $\mathbf{r}'$ (eq. (25)). Given the state comprises orientation, biases, velocity, position, extrinsics, and camera pose, each of these is updated according to $\delta\mathbf{x}$ (see eqs. (26) to (29)).

$$\delta\mathbf{x} = \mathbf{K}\mathbf{r}' \quad (25)$$

$$\delta\mathbf{q} \approx \begin{bmatrix} 1 \\ \frac{1}{2}\delta\boldsymbol{\theta} \end{bmatrix} \quad (26)$$

$$\begin{aligned} \mathbf{q} &\leftarrow \delta\mathbf{q} \otimes \mathbf{q} \\ \mathbf{b}_g &\leftarrow \mathbf{b}_g + \delta\mathbf{b}_g \\ \mathbf{v} &\leftarrow \mathbf{v} + \delta\mathbf{v} \end{aligned} \quad (27)$$

$$\begin{aligned} \mathbf{p} &\leftarrow \mathbf{p} + \delta\mathbf{p} \\ \mathbf{R}_I^C &\leftarrow \delta\mathbf{R}\mathbf{R}_I^C \\ \mathbf{t}_C^I &\leftarrow \mathbf{t}_C^I + \delta\mathbf{t}_C^I \end{aligned} \quad (28)$$

$$\begin{aligned} \mathbf{p}_{C_i} &\leftarrow \mathbf{p}_{C_i} + \delta\mathbf{p}_{C_i} \\ \mathbf{q}_{C_i} &\leftarrow \delta\mathbf{q}_{C_i} \otimes \mathbf{q}_{C_i} \end{aligned} \quad (29)$$

Lastly, state covariance matrix $\mathbf{P}$ is also updated and made symmetric (eqs. (30) and (31)).

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H}')\mathbf{P} \quad (30)$$

$$\mathbf{P} \leftarrow \frac{1}{2}(\mathbf{P} + \mathbf{P}^T) \quad (31)$$

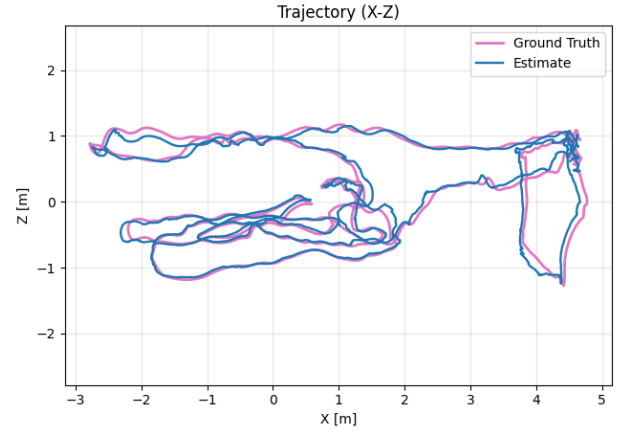


Fig. 2. Our estimated trajectory (blue) plotted with the ground truth (pink). View is of the X and Y axes.

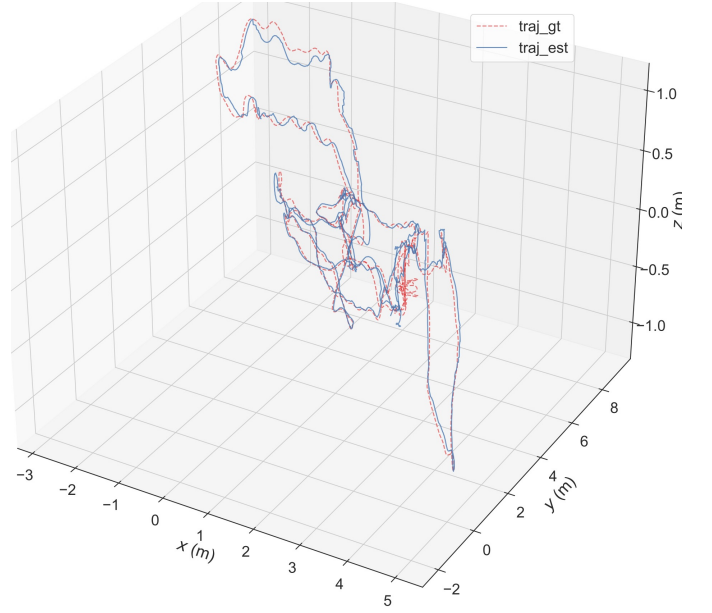


Fig. 3. Our estimated trajectory (blue) plotted with the ground truth (red). View is of the X, Y, and Z axes. Produced with [4].

II. RESULTS

Refer to Figures 2 and 3 for visual demonstrations of our final results. Presented alongside our RMSE RPE in Table I is our RMSE ATE of 0.078719. Both this value and our plots reflect the high quality of our implementation.

TABLE I
ABSOLUTE TRAJECTORY ERROR (ATE) AND RELATIVE POSE ERROR (RPE) AS CALCULATED WITH [4]

Error Metric	RSME
Absolute Trajectory (ATE)	0.078719
Relative Pose (RPE)	0.009437

III. CONCLUSION

In this project, we implemented the Stereo Multi-State Constraint Kalman Filter from Sun et al [1], and visualized it through a series of graph, evaluating its performance through RSME, Absolute Trajectory Error. This filter uses an IMU to enhance the accuracy of the camera pose estimate by providing high-frequency motion tracking, which compensates for the camera's poor accuracy in environments of motion blur. In addition, vision provides drift correction to the IMU readings. The combination results on a most robust estimate, evident in the trajectory comparison with the EuRoC dataset [3]. This was validated further with error calculations, which showed that the absolute trajectory had an error of 0.078719 while the relative pose only errored by 0.009437.

IV. CONTRIBUTION STATEMENT

Pau Alcolea: Implementation, Visualization, Writing, **Ben Orr:** Implementation, Visualization, Writing,

REFERENCES

- [1] K. Sun, K. Mohta, B. Pfrommer, M. Watterson, S. Liu, Y. Mulgaonkar, C. J. Taylor, and V. Kumar, "Robust stereo visual inertial odometry for fast autonomous flight," 2018. [Online]. Available: <https://arxiv.org/abs/1712.00036>
- [2] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint kalman filter for vision-aided inertial navigation," in *Proceedings 2007 IEEE International Conference on Robotics and Automation*, 2007, pp. 3565–3572.
- [3] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The euroc micro aerial vehicle datasets," *The International Journal of Robotics Research*, 2016. [Online]. Available: <http://ijr.sagepub.com/content/early/2016/01/21/0278364915620033.abstract>
- [4] M. Grupp, "evo: Python package for the evaluation of odometry and slam." <https://github.com/MichaelGrupp/evo>, 2017.